

Memory Management

Theory for the cvlab practise tasks

Everything needed to solve tasks 1–7 in `memory_management/practise.md`, plus the extras the mini real-time frame pipeline adds. Written against this repo's conventions: C++20, `-Wall -Wextra -Wconversion`, GoogleTest, and the `debug / asan / tsan` presets.

ON YOUR MACHINE

Every claim marked **ON YOUR MACHINE** in this document was measured here, not recalled from documentation: macOS 25.6 (Darwin), Apple Silicon arm64, Apple clang 21.0.0, libc++. Several of them contradict what the task descriptions assume, so read those boxes before you start a task — they will save you an hour of confusion each.

2026-09-24

Contents

0. How to use this	4
1. The memory model	5
1.1 Where things live	5
1.2 Why the stack is so much faster	5
1.3 The measurement hazards	6
1.4 Stack limits	6
2. Value categories and move semantics	8
2.1 Value categories	8
2.2 <code>std::move</code> moves nothing	8
2.3 The rule of five, and the trap inside it	8
2.4 What a correct move leaves behind	9
2.5 <code>noexcept</code> , and the experiment that proves it	9
2.6 Copy elision will hide your prints	10
3. Perfect forwarding	11
3.1 Forwarding references	11
3.2 Reference collapsing	11
3.3 <code>std::forward</code>	11
3.4 The greedy constructor	12
4. Dangling pointers and use-after-free	13
4.1 What actually goes wrong	13
4.2 How AddressSanitizer works	13
4.3 What ASan catches by default — and what it does not	13
4.4 Reading the report	14
5. Memory leaks	15
5.1 Leaks and near-leaks	15
5.2 The leak you will actually write	15
5.3 The tools — and where they run	15
6. Memory pools	17
6.1 Why bother	17
6.2 The free list	17
6.3 Constructing objects in pool memory	17
6.4 The RAI handle	18
6.5 Benchmarking it honestly	18

7. Alignment	19
7.1 The rules	19
7.2 Three ways to get aligned memory	19
7.3 Two things task 7 assumes that are false here	19
7.4 The SIMD step, on arm64	20
8. What the pipeline project adds	21
8.1 std::span	21
8.2 The ring buffer	21
8.3 Threads, mutexes and condition variables	21
8.4 The bug you introduce on purpose	22
9. Command cheat sheet	23
Assertions worth keeping in your tests	23
Platform matrix	23

0. How to use this

Each chapter maps onto one practise task. Read the chapter, then write the test. The theory is deliberately front-loaded with the things that are invisible until they bite you: what the compiler is allowed to delete, what the sanitizer will not tell you, and where macOS and Linux disagree.

Chapter	Task	Core idea
1. Memory model	1 — stack vs heap	Where objects live, and why one allocator is 20x the other
2. Move semantics	2 — Buffer	Value categories, rule of five, why <code>noexcept</code> changes behaviour
3. Perfect forwarding	3 — factory	Forwarding references, reference collapsing, <code>std::forward</code>
4. Lifetime bugs	4 — dangling / UAF	UB, ASan's shadow memory, reading a report
5. Leaks	5 — leak hunt	Exception safety, LeakSanitizer vs Valgrind
6. Pools	6 — free-list pool	Free lists, placement new, honest benchmarking
7. Alignment	7 — alignment	<code>alignas</code> , over-aligned new, SIMD loads
8. Pipeline extras	integration project	<code>std::span</code> , ring buffers, threads and condition variables

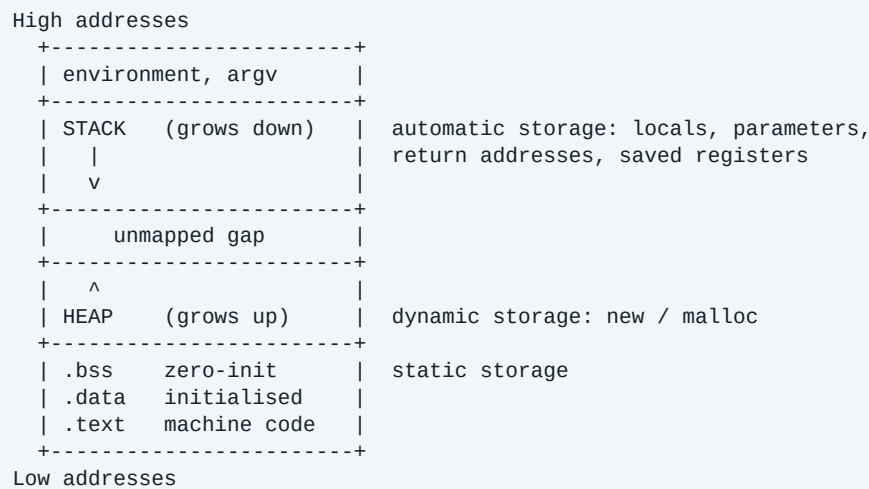
LIFEHACK

The repo's TDD rule (write the test, watch it fail) has a practical edge here: for half these tasks the first failure is a **link error**, not an assertion. The placeholder headers declare the classes but define nothing, so `undefined symbol: cvlab::memory::Buffer::Buffer` is the expected first result. That still counts as watching it fail.

1. The memory model

1.1 Where things live

A running process has its address space carved into regions. Only two matter for task 1:



C++ does not talk about stack and heap; it talks about **storage duration**. The mapping is what you would expect:

Storage duration	Created by	Lives until	Typically
automatic	a local variable	end of enclosing scope	stack
dynamic	new / malloc	delete / free	heap
static	globals, static locals	program exit	.data / .bss
thread	thread_local	thread exit	per-thread block

1.2 Why the stack is so much faster

Stack allocation is pointer arithmetic performed by the compiler. Reserving 1 KB of locals is one instruction, and releasing it is one more:

```
; int st[256];    ->   reserve 1024 bytes in the frame
    sub  sp, sp, #1024    ; allocate
    ...
    add  sp, sp, #1024    ; deallocate - that is the entire cost
```

There is no bookkeeping, no search, no lock, and no failure mode short of overflowing the whole stack. The memory is also almost certainly already in L1 cache, because the same few kilobytes near the stack pointer are reused by every call.

Heap allocation is a function call into a general-purpose allocator:

```

new int[256]
-> operator new(1024)
    -> malloc(1024)
        pick a size class for 1024 bytes
        take a lock, or consult this thread's cache
        pop a block off that size class's free list
        (or carve from the arena, or mmap if large)
        write bookkeeping metadata for the block
-> return the pointer

delete[] p
-> operator delete(p)
    -> free(p)
        read the block metadata
        possibly coalesce with neighbouring free blocks
        push back onto a free list

```

So the heap pays for: a non-inlined call, a branch-heavy size-class lookup, synchronisation, a pointer chase into metadata that is often cold in cache, and — on first touch of fresh pages — a page fault into the kernel.

ON YOUR MACHINE

Your task 1 implementation measures this. Ten million iterations allocating 256 ints: **stack 13 ms, heap 258 ms** — very close to 20x. The exact ratio is not the point; the order of magnitude is.

1.3 The measurement hazards

A benchmark of allocation is unusually easy to get wrong, because both the compiler and the allocator conspire to make your loop disappear.

- **Dead code elimination.** An unused stack array is not allocated at all; the compiler simply does not reserve the space. Your loop must write to the array and the result must be observable, or you are timing an empty loop.
- **Allocation elision.** Since C++14 the compiler is explicitly allowed to remove or merge `new` / `delete` pairs. At `-O2` a `new` immediately followed by `delete` can vanish entirely.
- **First-touch page faults.** Freshly mapped pages are not backed by physical memory until written. The first pass through a large buffer pays kernel time that later passes do not, so a cold first iteration skews short benchmarks.
- **Allocator warm-up.** The very first allocation in a process sets up arenas. Discard it.

LIFEHACK

This repo links Google Benchmark for exactly this reason. `benchmark::DoNotOptimize(p)` forces the compiler to assume `p` escapes, and `benchmark::ClobberMemory()` forces pending writes to memory. `common/tests/benchmark_smoke_test.cpp` already shows both. Use them in `frame_pool_bench.cpp` for task 6 — without them your pool will look infinitely fast, which is a bug in the benchmark, not a win.

1.4 Stack limits

The stack is fast but small, and it is not the same size everywhere. This matters directly for the pipeline project, where the ring buffer holds its storage in a `std::array` member.

ON YOUR MACHINE

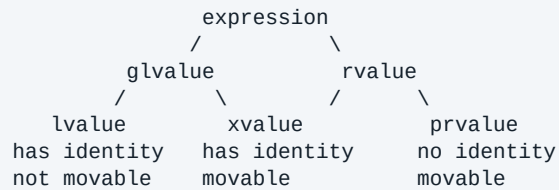
Measured here: **main thread 8176 KB, but a `std::thread` gets only 524 KB** — a 16x difference. A `RingBuffer<PooledFrame, N>` declared as a local inside a thread function is on that smaller stack. With large frames and a large `N` you will overflow it, and a stack overflow in a secondary thread is a bare crash with no message. Keep the ring buffer itself outside the thread and pass a reference, which is what a producer/consumer design wants anyway.

Check the limit with `ulimit -s`; a thread's own size comes from `pthread_attr_setstacksize` before it starts.

2. Value categories and move semantics

2.1 Value categories

Every expression has a type and a **value category**. Two independent questions define the taxonomy: does it have identity (can you take its address?), and can it be moved from?



Category	Example	Rule of thumb
lvalue	<code>b</code> , <code>v[0]</code> , <code>*p</code> , <code>f()</code> returning <code>T&</code>	It has a name or an address you can take
prvalue	<code>Buffer(8)</code> , <code>42</code> , <code>f()</code> returning <code>T</code>	A fresh temporary that has not been given a name
xvalue	<code>std::move(b)</code> , <code>static_cast<T&&>(b)</code>	A named object you have marked as expiring

TRAP

Inside `Buffer(Buffer&& other)`, the parameter `other` is an **lvalue** — it has a name. `T&&` describes how the argument was bound, not what the parameter now is. So `data_ = other.data_` copies, and if you forward it onward you must write `std::move(other)` again. Forgetting this is the single most common move-semantics bug.

2.2 `std::move` moves nothing

It is a cast, it generates no instructions, and it is roughly this:

```
template <typename T>
constexpr std::remove_reference_t<T>&& move(T&& t) noexcept {
    return static_cast<std::remove_reference_t<T>&&>(t);
}
```

All it does is change the value category of the expression, so that overload resolution picks the `T&&` overload instead of the `const T&` one. The actual stealing of resources happens in the move constructor you write.

2.3 The rule of five, and the trap inside it

Special member	Signature
destructor	<code>~Buffer()</code>
copy constructor	<code>Buffer(const Buffer&)</code>
copy assignment	<code>Buffer& operator=(const Buffer&)</code>
move constructor	<code>Buffer(Buffer&&) noexcept</code>
move assignment	<code>Buffer& operator=(Buffer&&) noexcept</code>

The compiler generates these for you, but declaring some of them suppresses others. This table is the part people get wrong:

If you declare...	then you get...
nothing	all five, implicitly

If you declare...	then you get...
a destructor	copies still implicit, but moves are NOT generated — every 'move' silently becomes a copy
a copy operation	moves are NOT generated ; the other copy op is implicit (deprecated)
a move operation	both copy operations are deleted

TRAP

The destructor row is the killer. A class that manages a raw pointer needs a destructor — and the moment you write one, you have silently disabled move semantics for the whole type. Everything still compiles and still works; it is just quietly copying. This is why the rule is *five*, not *one*: declare all of them or none of them.

For task 2 you are declaring all five anyway, which is the point of the exercise. In production code the better answer is usually the rule of **zero**: hold a `std::unique_ptr` or a `std::vector` and let the compiler generate everything correctly.

2.4 What a correct move leaves behind

The standard requires a moved-from object of a library type to be in a state that is **valid but unspecified**: you may destroy it or assign to it, but you may not assume what it holds. For your own types you decide, and the only sane choice is a state your destructor survives:

```
Buffer::Buffer(Buffer&& other) noexcept
    : data_(other.data_), size_(other.size_) {
    other.data_ = nullptr;      // <- mandatory: else double delete
    other.size_ = 0;
}

Buffer& Buffer::operator=(Buffer&& other) noexcept {
    if (this == &other) return *this;    // self-assignment
    delete[] data_;                      // release what we already own
    data_ = std::exchange(other.data_, nullptr);
    size_ = std::exchange(other.size_, 0);
    return *this;
}
```

- Null the source. Without it, both objects destroy the same buffer.
- Release your own resource first in *assignment* (not in the constructor, where there is nothing to release yet).
- `std::exchange` from `<utility>` is the tidy way to say 'take the value and leave a default behind'.
- `delete[] nullptr` is well defined and does nothing, so you do not need to guard it.

2.5 noexcept, and the experiment that proves it

`std::vector` must give the **strong exception guarantee** on reallocation: if anything throws while moving elements to the new block, the vector must look untouched. But a move that throws has already gutted some source elements — it cannot roll back. A copy that throws is harmless, because the originals are intact.

So `std::vector` uses `std::move_if_noexcept`: it moves only if the move constructor is `noexcept` (or if the type has no copy constructor at all). Otherwise it copies, and your careful move semantics do nothing.

ON YOUR MACHINE

Verified here with `libc++`, pushing four elements and watching reallocation. Output condensed:

<pre> ### move ctor IS noexcept push_back #1 (size=1 cap=1) ctor move ctor <- argument move ctor <- relocating -> size=2 cap=2 push_back #2 (size=2 cap=2) ctor move ctor move ctor <- relocating move ctor -> size=3 cap=4 </pre>	<pre> ### move ctor is NOT noexcept push_back #1 (size=1 cap=1) ctor move ctor <- argument COPY ctor <- relocating! -> size=2 cap=2 push_back #2 (size=2 cap=2) ctor move ctor COPY ctor <- relocating! COPY ctor -> size=3 cap=4 </pre>
--	--

Two things to read out of that. First, the move right after `ctor` is the temporary argument being moved into the slot — that happens either way. The difference is only in **relocating the existing elements**. Second, the capacity goes 0, 1, 2, 4: `libc++` grows by 2x, so reallocation happens on pushes 1, 2 and 3 but not 4.

LIFEHACK

Assert it instead of hoping: `static_assert(std::is_nothrow_move_constructible_v<Buffer>);` That one line catches the whole family of mistakes — a forgotten `noexcept`, a member whose own move can throw, and the destructor trap from 2.3 — at compile time. Put it in your test.

LIFEHACK

To see the copies, run the test binary directly rather than through `ctest`, which swallows stdout on success:
`./build/debug/part1_modern_cpp/memory_management/cvlab_memory_management_test`
`--gtest_filter='Buffer.*'`

2.6 Copy elision will hide your prints

Since C++17, initialising an object from a prvalue of the same type does not call any constructor — the object is constructed directly in place. This is **guaranteed elision**, not an optimisation, and it cannot be turned off:

```

Buffer make() { return Buffer(8); } // no move, no copy - ever
Buffer b = make();                // still just one ctor call

```

Named return value optimisation (returning a local variable by name) is a separate, *optional* optimisation, and `-fno-elide-constructors` disables that one if you want to see the moves that would otherwise happen.

TRAP

Never write `return std::move(local);`. It turns the return expression into an xvalue, which **disables NRVO** and forces an actual move where you would have got nothing at all. Returning a local by name already moves when it can. Compilers warn about this under `-Wpessimizing-move`, which `-Wall` enables here.

3. Perfect forwarding

3.1 Forwarding references

T&& means 'rvalue reference' almost everywhere — except when T is a template parameter being deduced **by that same function**, where it becomes a *forwarding reference* that binds to anything:

```
template <typename T> void f(T&& x);    // forwarding reference

int i = 0;
f(i);      // argument is an lvalue  -> T = int&
f(42);     // argument is a prvalue  -> T = int
```

auto&& x = expr; is the other spelling of the same thing.

TRAP

These are **not** forwarding references, because nothing is being deduced at the call: `void g(Buffer&& b);` — concrete type. `template <typename T> class Pool { void push(T&& v); };` — T was fixed when the class was instantiated. To forward from a member of a class template you need the member itself to be a template: `template <typename U> void push(U&& v);`

3.2 Reference collapsing

Deduction above produced `int& &&`, which is not a real type. The language collapses such pairs with one rule: **an lvalue reference wins**.

Written	Collapses to
T& &	T&
T& &&	T&
T&& &	T&
T&& &&	T&&

So `f(i)` deduces `T = int&`, the parameter type `T&&` becomes `int& && = int&`, and the lvalue binds to an lvalue reference. `f(42)` deduces `T = int`, the parameter is `int&&`, and the temporary binds to an rvalue reference. One function, both cases, no overloads.

3.3 std::forward

The parameter is now an lvalue regardless of what was passed (2.1), so passing it on would always select the copying overload. `std::forward` restores the original category by casting back to `T&&` and letting the collapsing rules do the work:

```
template <typename T>
constexpr T&& forward(std::remove_reference_t<T>& t) noexcept {
    return static_cast<T&&>(t);
}

// T = int&  -> static_cast<int& &&>  -> int&    : stays an lvalue
// T = int   -> static_cast<int&&>    -> int&&   : becomes an rvalue
```

Which gives the whole idiom — this is task 3:

```
template <typename T, typename... Args>
T create(Args&&... args) {
    return T(std::forward<Args>(args)...);
}
```

- `Args&&... args` — a pack of forwarding references.
- `std::forward<Args>(args)...` — the pack expands so that each argument is forwarded with its own category. The `...` applies to the whole pattern.

TRAP

The explicit `<Args>` is mandatory. `std::forward` deliberately cannot deduce its template argument — deducing it would defeat the entire mechanism — so `std::forward(args)` does not compile. If you find yourself wanting `std::forward` without a template argument, you wanted `std::move`.

TRAP

Forward exactly once. A forwarded argument may have been moved from, so using the pack twice is a use-after-move: `T(std::forward<Args>(args)..., std::forward<Args>(args)...) — the second expansion sees gutted objects.`

LIFEHACK

To prove the right overload ran, give your test class two constructors that record which one fired, then assert on that. Then break it on purpose: change `std::forward<Args>(args)...` to plain `args...` and watch the rvalue test start selecting the `const&` overload. Seeing the failure is the lesson; the working version is three tokens.

3.4 The greedy constructor

A forwarding constructor will out-compete your copy constructor, because an exact match beats a conversion:

```
struct Widget {
    template <typename T>
    explicit Widget(T&& name);           // matches Widget& exactly...
    Widget(const Widget&);               // ...better than this does
};

Widget a{"x"};
Widget b{a};    // calls the TEMPLATE, not the copy constructor
```

The fix in C++20 is a constraint:

```
template <typename T>
    requires (!std::same_as<std::remove_cvref_t<T>, Widget>)
explicit Widget(T&& name);
```

4. Dangling pointers and use-after-free

4.1 What actually goes wrong

An object's lifetime ends when its destructor runs or its storage is released. A pointer that outlives the object it points to is **dangling**, and reading or writing through it is undefined behaviour.

Undefined behaviour does not mean 'crashes'. It means the compiler is entitled to assume it never happens, and to optimise on that assumption. The practical consequences are worse than a crash: the program may appear to work for months, then change behaviour when you raise the optimisation level or add an unrelated function.

```
int* dangling_pointer() {
    int x = 42;
    return &x;          // x dies here; the returned address is garbage
}

void use_after_free() {
    int* p = new int(42);
    delete p;
    *p = 7;              // the allocator may already have reused this block
}
```

LIFEHACK

The compiler catches the easy one for free. This repo already builds with `-Wall -Wextra`, and clang emits *'address of stack memory associated with local variable x returned'* (`-Wreturn-stack-address`) for the first function. Read your build output before reaching for a sanitizer — it is already telling you.

4.2 How AddressSanitizer works

ASan instruments the program at compile time. Two mechanisms:

- **Shadow memory.** Every 8 bytes of application memory maps to 1 shadow byte recording how many of those 8 bytes are addressable. Every load and store gets a shadow check compiled in front of it. This is why ASan costs roughly 2x runtime and a large chunk of address space, rather than the 20–50x of an interpreter-based tool.
- **Redzones and quarantine.** Allocations are padded with poisoned redzones so overflow lands in poison rather than in a neighbour, and freed blocks are held in quarantine rather than immediately reused, so a use-after-free hits poison instead of a recycled block.

4.3 What ASan catches by default — and what it does not

Check	On by default?
heap-buffer-overflow	yes
heap-use-after-free	yes
stack-buffer-overflow	yes
global-buffer-overflow	yes
stack-use-after-scope	yes
stack-use-after-return	NO — needs <code>ASAN_OPTIONS=detect_stack_use_after_return=1</code>
memory leaks	Linux only — see chapter 5

ON YOUR MACHINE

Verified here, and this is the trap in task 4. Compiled with `-fsanitize=address -O0 -g`, the dangling-stack-pointer program ran **cleanly and printed 42**, exit status 0. ASan said nothing. With `ASAN_OPTIONS=detect_stack_use_after_return=1` the same binary produced a full report. If you skip this, you will conclude ASan cannot catch the bug — it can, it is just switched off.

4.4 Reading the report

The real output from that run, annotated:

```
==9598==ERROR: AddressSanitizer: stack-use-after-return on address    <- what
    0x000106eb5020 at pc 0x000104b9ca5c ...
READ of size 4 at 0x000106eb5020 thread T0                            <- how
    #0 0x000104b9ca58 in main probe_dangle.cpp:10                    <- WHERE
    #1 0x000180e144e0 in start+0x1b4c

Address 0x000106eb5020 is located in stack of thread T0 at offset 32
in frame
    #0 0x000104b9c84c in dangling_pointer() probe_dangle.cpp:3      <- origin
This frame has 1 object(s):
    [32, 36) 'x' (line 4) <== Memory access at offset 32 is inside
                                this variable                      <- which
```

- Line 1 names the **bug class**. Learn these names; they are precise.
- The first stack trace is where the *bad access* happened.
- The second block is where the memory *came from* — for a heap UAF you get both the allocation and the deallocation stack, which is usually the information you actually needed.
- The last line names the variable and its offset.

LIFEHACK

Three practical flags. `ASAN_OPTIONS=abort_on_error=1` (this repo's CI already sets it) turns the report into a `SIGABRT` so a test harness notices. `halt_on_error=0` reports and continues, useful for collecting several bugs in one run. And build at `-O0` or `-O1`: at `-O2` the compiler may delete a pointless read entirely, leaving nothing for ASan to check.

LIFEHACK

Run the sanitized binary **directly**, not through `ctest`. `ctest` hides output for passing tests, and an ASan report on a test that otherwise passes is easy to miss. If you do use `ctest`, pass `--output-on-failure`.

5. Memory leaks

5.1 Leaks and near-leaks

A leak is memory that is still allocated at exit with nothing pointing at it. The more useful distinction tools draw is whether a pointer to the block still exists somewhere:

Valgrind term	LeakSanitizer term	Meaning
definitely lost	Direct leak	No pointer to the block survives. A real bug.
indirectly lost	Indirect leak	Reachable only from a block that is itself lost — e.g. the children of a leaked tree node. Fix the parent.
possibly lost	(not reported separately)	Only an interior pointer survives. Often fine, sometimes a bug.
still reachable	(not reported by default)	A live pointer exists at exit — a global cache, a singleton. Untidy, usually harmless.

ON YOUR MACHINE

This is why this repo's CI runs ASan with `detect_leaks=0`: OpenCV's static initialisers allocate blocks that are never freed. They are *still reachable*, not leaks in your code. [docs/setup.md](#) documents this.

5.2 The leak you will actually write

Overwriting the only pointer is the obvious one. The interesting one is the exception path, because it is invisible in the source:

```
void f() {
    int* p = new int[100];
    might_throw();           // if this throws, delete[] never runs
    delete[] p;              // and nothing in the source looks wrong
}

void g() {
    auto p = std::make_unique<int[]>(100);
    might_throw();           // ~unique_ptr runs during stack unwinding
}                             // no leak, and no delete to forget
```

This is the entire argument for RAII in one page: `f` is correct only on the happy path, and every early return you add later reintroduces the bug. `g` cannot leak.

5.3 The tools — and where they run

ON YOUR MACHINE

Neither leak tool from task 5 works on your Mac. Both were checked here: • `ASAN_OPTIONS=detect_leaks=1` aborts immediately with *'detect_leaks is not supported on this platform'* (exit 134). LeakSanitizer has no arm64 macOS support. • Valgrind is not installed and cannot be: it has no macOS arm64 port at all. Task 5's comparison has to happen on Linux — your Ubuntu machine, CI, or Docker. Note this also means the `ASAN_OPTIONS=detect_leaks=1` suggestion in your own `docs/setup.md` only holds on Linux.

The quickest local route is a container. Valgrind supports arm64 Linux, so this runs natively on Apple Silicon with no emulation:

```
docker run --rm -it -v "$PWD":/src -w /src ubuntu:24.04 bash
apt-get update
apt-get install -y build-essential cmake ninja-build valgrind
# then the normal cvlab commands, plus:
valgrind --leak-check=full --show-leak-kinds=all \
    ./build/debug/.../cvlab_memory_management_test \
    --gtest_filter='MemoryLeak.*'
```

What each tool is for, so you pick the right one rather than the familiar one:

Tool	Catches	Cost	Platform
ASan (+LSan)	overflows on heap/stack/globals, UAF, double free, leaks	~2x, needs recompile	LSan: Linux only
Valgrind memcheck	leaks, UAF, uninitialised reads , bad frees	20–50x, no recompile	Linux (incl. arm64)
MSan	uninitialised reads, precisely	~3x, needs <i>everything</i> rebuilt	Linux/clang
UBSan	signed overflow, bad shifts, misaligned access, bad casts	small	both
TSan	data races	5–15x	both (see caveat)

LIFEHACK

The most useful difference to internalise: **ASan does not detect uninitialised reads at all**. That is Valgrind's or MSan's job. If a value is mysteriously wrong but ASan is silent, you are looking at the wrong tool, not at a clean program. And ASan and Valgrind must never be combined — they both rewrite memory management and will fight.

LIFEHACK

You can prove your leak exists without any sanitizer. Your task 1 code already replaces global `operator new`; the same trick counts allocations against deallocations inside a test and asserts the balance. That is portable, runs on the Mac, and is often the fastest feedback loop.

6. Memory pools

6.1 Why bother

Chapter 1 established that a general-purpose allocator does real work per call. A pool trades generality for speed: fixed-size blocks, allocated up front, handed out and returned in constant time.

- **Bounded latency.** A real-time frame pipeline cannot afford a `malloc` that occasionally takes a slow path, grabs a lock, or faults in new pages. The worst case matters more than the average.
- **No fragmentation.** All blocks are the same size, so a free block is always usable. General allocators can fragment until a large request fails despite plenty of free memory.
- **Cache behaviour.** A pool's blocks are contiguous and reused, so they stay warm. Fresh heap allocations wander.
- **Backpressure.** A fixed pool that runs dry tells you the consumer is too slow. An unbounded allocator just grows until something worse happens.

6.2 The free list

The trick that makes a pool nearly free: a block on the free list has no live contents, so store the linked-list pointer **inside the block itself**. Zero bookkeeping overhead.

```
union Slot {
    Slot* next;           // while free: the free-list link
    std::byte storage[kSize]; // while in use: the payload
};

void* acquire() {         // 0(1), no branches beyond the empty check
    if (!head_) return nullptr;
    Slot* s = head_;
    head_ = head_->next;
    return s;
}

void release(void* p) {   // 0(1)
    Slot* s = static_cast<Slot*>(p);
    s->next = head_;
    head_ = s;
}
```

A `std::vector<void*>` of free pointers is a perfectly good way to start — simpler to debug, one extra indirection. Write that first, then switch to the intrusive version once the tests pass.

6.3 Constructing objects in pool memory

A pool hands you raw storage. To put an object in it you need **placement new**, and you must then destroy it by hand:

```
void* raw = pool.acquire();
T* obj = new (raw) T(args...); // construct in place, no allocation
...
obj->~T();                     // explicit destructor call
pool.release(raw);             // storage back to the free list
```

TRAP

Never `delete obj` after placement new. `delete` calls the destructor *and* frees the storage with `operator delete` — but that storage came from your pool, not from the global allocator. The result is heap corruption. Destructor by hand, storage back to the pool.

6.4 The RAIL handle

`PooledFrame` exists so that step cannot be forgotten. It owns a slot and a pointer back to its pool, returns the slot in its destructor, and is move-only — two handles owning one slot would double-release.

```
PooledFrame::~PooledFrame() {
    if (pool_ && data_) pool_>release(data_);    // guard: may be moved-from
}

PooledFrame::PooledFrame(PooledFrame&& o) noexcept
    : pool_(std::exchange(o.pool_, nullptr)),
      data_(std::exchange(o.data_, nullptr)) {}
```

- The destructor must tolerate a moved-from handle — hence the null check. This is the same discipline as 2.4.
- Copy operations are = `deleted` in the placeholder header already. That is what makes the ring buffer's push/pop provably copy-free later.
- A handle must not outlive its pool. Destruction order in your test matters: declare the pool before anything that borrows from it, so it is destroyed last.

6.5 Benchmarking it honestly

Section 1.3 applies with full force. Additional pool-specific traps:

- **Measure steady state.** The first `acquire` touches pages for the first time. Warm up, then measure.
- **Compare like with like.** `new/delete` of the same size, in the same pattern. Allocating N then freeing N is a very different workload from alternating acquire/release, and the pool's advantage differs between them — measure both.
- **Do not let it vanish.** `benchmark::DoNotOptimize` on the pointer, every iteration.

LIFEHACK

Expect a smaller win than chapter 1's 20x. Modern allocators are good at exactly this pattern — same size, immediately reused, single-threaded, hot in the thread cache. If your pool is only 2–4x faster, that is a normal result, not a broken benchmark. The pool's real argument is the *worst case* and the absence of locking, which a single-threaded average does not show. Measuring the maximum acquire time rather than the mean makes the point far better.

7. Alignment

7.1 The rules

Every type has an alignment: the addresses at which an object of that type may start must be a multiple of it. `alignof(T)` reports it, `alignas(N)` raises it.

- **Natural alignment** is normally the type's size for scalars: 4 for `int`, 8 for `double`. A struct's alignment is the largest of its members'.
- **Over-alignment** is anything stricter than the default, which is what SIMD wants: 32 bytes for AVX's 256-bit registers, 64 for AVX-512.
- Hardware cares because a load that straddles a cache line or a page costs extra, and because some instructions require it outright.

ON YOUR MACHINE

Measured here: `__STDCPP_DEFAULT_NEW_ALIGNMENT__` is **16**. So plain `new` guarantees only 16-byte alignment — enough for any built-in type, not enough to *promise* 32 for AVX. Also: the cache line is **128 bytes** on Apple Silicon (`sysctl hw.cachelinesize`), twice the 64 bytes you get on x86_64, and `libc++` reports `hardware_destructive_interference_size` as 256. Anything you pad to avoid false sharing needs the larger number on this machine.

7.2 Three ways to get aligned memory

```
// 1. alignas on the type - C++17 routes new/delete to the aligned
//    overloads automatically. Portable, no size rules, no mismatch.
struct alignas(32) Frame { float px[64]; };
Frame* f = new Frame;      // operator new(size, align_val_t{32})
delete f;                  // matching aligned operator delete

// 2. std::aligned_alloc - C API rules apply (see the trap below)
auto* p = static_cast<float*>(std::aligned_alloc(32, 64 * sizeof(float)));
std::free(p);              // MUST be free(), never delete[]

// 3. Over-allocate and round the pointer up by hand - what you do
//    when the first two are unavailable. Keep the original pointer
//    to free it.
```

LIFEHACK

Option 1 is the one to reach for. `alignas` on the type means the compiler picks the right `operator new` and the right `operator delete` for you, there is no size-multiple rule to remember, and there is no way to free it wrongly. `aligned_alloc` is worth writing once, for task 7, so you know what it costs.

TRAP

`std::aligned_alloc` is paired with `std::free`. Calling `delete[]` on it, or `free` on a `new[]` pointer, is undefined behaviour that often 'works' until it corrupts the heap under load.

7.3 Two things task 7 assumes that are false here

ON YOUR MACHINE

The **size rule is enforced on macOS and not on glibc**. The C standard says the size passed to `aligned_alloc` must be a multiple of the alignment. Measured here: `std::aligned_alloc(32, 100)` returns `nullptr`. `glibc` is lenient and typically succeeds with the same call. So code that works on your Ubuntu box can return null on the Mac — round the size up yourself: `size = ((size + align - 1) / align) * align;`

ON YOUR MACHINE

Plain new was 32-byte aligned every single time. Measured with 16 simultaneously-live allocations: `new float[64]` gave 16/16 32-byte-aligned pointers, and `new float[13]` also 16/16. macOS's allocator simply hands out generously aligned blocks for these size classes. This breaks the obvious test.

`EXPECT_FALSE(is_aligned(plain_new_ptr, 32))` will **fail on this machine**, and would be wrong anyway — nothing forbids an allocator from being more aligned than required. Assert what the standard actually promises: the `aligned_alloc` buffer is 32-aligned (guaranteed), and the plain buffer is 16-aligned (guaranteed). Whether the plain one is *also* 32-aligned is luck, and your test should not depend on it either way.

LIFEHACK

When you need a genuinely misaligned pointer — for the SIMD experiment, or to test `is_aligned` returns false — do not hope for one. Manufacture it: `auto bad =`

```
reinterpret_cast<float>(reinterpret_cast<char*>(buf) + 4);
```

7.4 The SIMD step, on arm64

ON YOUR MACHINE

`_mm256_load_ps` **does not exist on your machine**. AVX, SSE and `<immintrin.h>` are x86-only; this is arm64. The optional part of task 7 cannot run here as written.

What to do instead, in order of value:

- **Understand the x86 rule without running it.** `_mm256_load_ps` *requires* 32-byte alignment and raises a general-protection fault otherwise; `_mm256_loadu_ps` is the unaligned form and does not. On x86, alignment is a **correctness** issue for the aligned intrinsics.
- **Run it on the Ubuntu x86_64 machine**, where the crash is real and instructive. Compile with `-mavx`.
- **Use NEON locally for contrast.** `<arm_neon.h>`, `vld1q_f32(ptr)`. AArch64 permits unaligned loads on normal memory, so it will *not* crash — on ARM, alignment is a **performance** issue, not a correctness one, for ordinary loads. Atomics and exclusive load/store still require alignment.
- **Catch it with a sanitizer instead of a crash.** UBSan's alignment check (`-fsanitize=alignment`, included in `-fsanitize=undefined`) diagnoses a misaligned typed access on either architecture, which is the portable way to see the bug.

LIFEHACK

Do not let this become a blocker. The transferable lesson of task 7 is *check alignment explicitly, and get your buffers aligned deliberately rather than by accident* — which is exactly what the pipeline project then needs. The crash is a demonstration, not the point.

8. What the pipeline project adds

8.1 `std::span`

A `std::span<T>` is a pointer and a length. It owns nothing, copies nothing, and is the C++20 way to say 'a contiguous run of T that somebody else owns'. That is precisely the zero-copy requirement: the consumer processes pool memory in place.

```
void process(std::span<uint8_t> frame);    // no ownership, no copy

PooledFrame f = pool.acquire();
process(std::span<uint8_t>{
    reinterpret_cast<uint8_t*>(f.data()), f.size()});
```

TRAP

A span is a borrowed view with no lifetime protection whatsoever. If the `PooledFrame` is destroyed — or moved from, returning its slot to the pool — while a span over it is still in use, that span dangles and chapter 4 applies. Never store a span in a member that outlives the call; pass it down, do not keep it.

LIFEHACK

To prove zero-copy in a test, capture `span.data()` before and after `process` and assert the pointer is unchanged and still inside the pool's address range. That is an assertion on a real value, which is what this repo asks for, and it is much stronger than asserting the contents changed.

8.2 The ring buffer

A bounded queue over a fixed `std::array`: a head index, a tail index, and either a count or a deliberately-wasted slot to tell full from empty. No allocation after construction, which is the whole point — an unbounded `std::deque` would reintroduce the allocator you just removed.

- Make `N` a power of two and replace `% N` with `& (N - 1)`. Division is slow; masking is one instruction.
- Push and pop by move. With `PooledFrame` this is enforced for you — it has no copy constructor, so a copy will not compile.
- Get it right single-threaded first, and only then add locking. Debugging index arithmetic and a condition variable at the same time is how weekends disappear.

8.3 Threads, mutexes and condition variables

```
void push(T v) {
    std::unique_lock lock(m_);
    not_full_.wait(lock, [this] { return count_ < N; });    // predicate!
    // ... store std::move(v), bump head_, ++count_
    lock.unlock();    // unlock before notifying: the woken
    not_empty_.notify_one();    // thread should not immediately block
}
```

- **Always use the predicate form of `wait`.** A condition variable may wake spuriously, with no notification at all. The predicate overload is exactly `while (!pred()) wait(lock);` and removes the entire class of bug.
- **Two condition variables**, not one: 'not full' for producers and 'not empty' for consumers. Sharing one means waking threads that cannot proceed.
- **Check the predicate under the lock.** That is why `wait` takes the lock and re-acquires it before returning.
- **Shutdown needs a plan.** A consumer blocked in `wait` will never return on its own. Add a `done_` flag, set it under the lock, and `notify_all()`; the predicate becomes `count_ > 0 || done_`. Otherwise your test hangs instead of failing, which is much worse.

- `std::jthread` (C++20) joins in its destructor and carries a `stop_token`. Prefer it — a `std::thread` that is neither joined nor detached calls `std::terminate`.

LIFEHACK

Write the multithreaded test so it *cannot hang forever*: have the producer push a fixed number of frames, and assert the consumer saw exactly that many. If you need a timeout, `wait_for` with a generous limit turns a hang into a failed assertion, which a CI run can actually report.

ON YOUR MACHINE

`docs/setup.md` already warns about TSan here, and it is worth repeating: third-party libraries are built uninstrumented, so TSan is trustworthy for **your** threading code and not for races involving OpenCV internals. For this project that is fine — the pipeline's threading is entirely yours. Run `cmake --preset tsan` and treat any report as real.

8.4 The bug you introduce on purpose

Step 6 of the pipeline practise asks you to break something and confirm the tools catch it. Choose deliberately — the instructive part is that the two failures look nothing alike:

Bug	What catches it	What it looks like
Drop a <code>std::move</code> on push	Nothing! It will not compile — <code>PooledFrame</code> has no copy ctor	A compile error, which is the good outcome: the type system prevented the bug
Return a <code>PooledFrame*</code> to a stack local	ASan, only with <code>detect_stack_use_after_return=1</code>	stack-use-after-return, per 4.3
Release a slot twice	ASan, or your own pool assertion	Two handles hand out the same buffer — silent data corruption until it isn't
Let a <code>PooledFrame</code> outlive its pool	ASan heap-use-after-free	The destructor calls into a destroyed pool

LIFEHACK

The first row is the most valuable thing in the exercise. A move-only type converts a runtime bug into a compile error. That is the entire design argument for RAII handles, and you get to see it rather than be told it.

9. Command cheat sheet

```
# --- build and test -----
cmake --preset debug && cmake --build --preset debug
ctest --preset debug          # all
ctest --preset debug -R MemoryManagement # one suite
ctest --preset debug --output-on-failure # show output

# --- run a binary directly (needed to SEE printf / ASan output) -----
B=./build/debug/part1_modern_cpp/memory_management
$B/cvlab_memory_management_test --gtest_filter='Buffer.*'
$B/cvlab_memory_management_test --gtest_list_tests
$B/cvlab_memory_management_bench          # google benchmark

# --- sanitizers -----
cmake --preset asan && cmake --build --preset asan
A=./build/asan/part1_modern_cpp/memory_management

# stack-use-after-return is OFF by default - task 4 needs this:
ASAN_OPTIONS=detect_stack_use_after_return=1 $A/cvlab_memory_management_test

ASAN_OPTIONS=abort_on_error=1 $A/cvlab_memory_management_test
ASAN_OPTIONS=detect_leaks=1 ... # Linux only; ABORTS on macOS arm64

cmake --preset tsan && cmake --build --preset tsan # data races

# --- leak hunting (Linux / Docker only) -----
docker run --rm -it -v "$PWD":/src -w /src ubuntu:24.04 bash
valgrind --leak-check=full --show-leak-kinds=all ./binary --gtest_filter=...

# --- seeing what the compiler really did -----
clang++ -std=c++20 -O0 -fno-elide-constructors f.cpp # show NRVO moves
clang++ -std=c++20 -S -O2 -o - f.cpp                # read the asm
```

Assertions worth keeping in your tests

```
static_assert(std::is_nothrow_move_constructible_v<Buffer>);
static_assert(!std::is_copy_constructible_v<PooledFrame>); // move-only
static_assert(std::is_nothrow_move_assignable_v<PooledFrame>);
static_assert(alignof(Frame) == 32);
EXPECT_EQ(reinterpret_cast<std::uintptr_t>(p) % 32, 0u);
```

Platform matrix

	macOS arm64 (this machine)	Ubuntu x86_64
ASan (overflow, UAF)	yes	yes
stack-use-after-return	yes, with <code>detect_stack_use_after_return=1</code>	same
LeakSanitizer	no — aborts, unsupported	yes
Valgrind	no — no arm64 macOS port	yes (also arm64 Linux)
TSan	yes (own code only)	yes (own code only)
AVX / <code>_mm256_load_ps</code>	no — x86 only	yes, with <code>-mavx</code>
NEON / <code>vld1q_f32</code>	yes	no
new default alignment	16	16

	macOS arm64 (this machine)	Ubuntu x86_64
<code>aligned_alloc</code> size not a multiple	returns nullptr	glibc usually succeeds
Cache line	128 bytes	64 bytes
<code>std::thread</code> stack	524 KB	typically 8 MB

The repo's own rule applies to your notes as much as to its commits: say which platform you actually ran on. Most of these tasks will only be exercised on one of the two, and CI is what settles the other.